

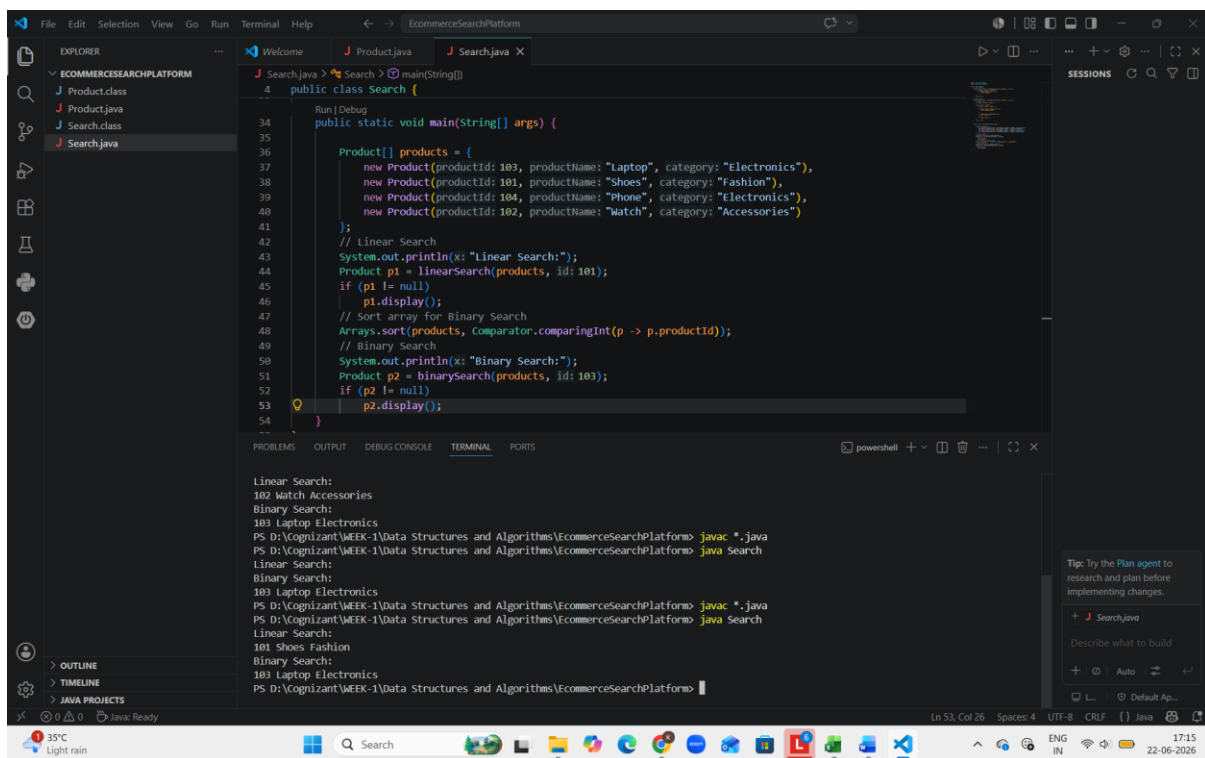
E-Commerce Search Platform

- ➔ Big O notation represents the complexity of an algorithm and performance as input size grows.
- ➔ It helps in measuring efficiency and comparing algorithms

Search cases:

- ➔ For best case the element is found immediately. Ex: Linear search- $O(1)$
- ➔ For average case the element is found somewhere in the middle. Ex: Linear search- $O(n/2)$
- ➔ For worst case the element is found at the last or not found. Ex: Linear search- $O(n)$

Output:



The screenshot shows an IDE with a Java project named 'EcommerceSearchPlatform'. The main file, 'Search.java', contains the following code:

```
public class Search {  
    public static void main(String[] args) {  
        Product[] products = {  
            new Product(productId: 103, productName: "Laptop", category: "Electronics"),  
            new Product(productId: 101, productName: "Shoes", category: "Fashion"),  
            new Product(productId: 104, productName: "Phone", category: "Electronics"),  
            new Product(productId: 102, productName: "Watch", category: "Accessories")  
        };  
        // Linear Search  
        System.out.println("Linear Search:");  
        Product p1 = linearSearch(products, 101);  
        if (p1 != null) {  
            p1.display();  
        }  
        // Sort array for Binary Search  
        Arrays.sort(products, Comparator.comparingInt(p -> p.productId));  
        // Binary Search  
        System.out.println("Binary Search:");  
        Product p2 = binarySearch(products, 103);  
        if (p2 != null) {  
            p2.display();  
        }  
    }  
}
```

The terminal output shows the results of the linear and binary searches:

```
Linear Search:  
102 Watch Accessories  
Binary Search:  
103 Laptop Electronics  
PS D:\Cognizant\WEEK-1\Data Structures and Algorithms\EcommerceSearchPlatform> javac *.java  
PS D:\Cognizant\WEEK-1\Data Structures and Algorithms\EcommerceSearchPlatform> java Search  
Linear Search:  
103 Laptop Electronics  
Binary Search:  
101 Shoes Fashion  
PS D:\Cognizant\WEEK-1\Data Structures and Algorithms\EcommerceSearchPlatform>
```

Time complexities of algorithms:

Linear search: Best case- $O(1)$, Average case- $O(n)$, Worst case- $O(n)$

Binary search: Best case- $O(1)$, Average case- $O(\log n)$, Worst case- $O(\log n)$

- ➔ Binary search is better for an e-commerce platform because of its efficiency, as e-com platforms have millions of product, it helps in faster search process.

Financial Forecasting:

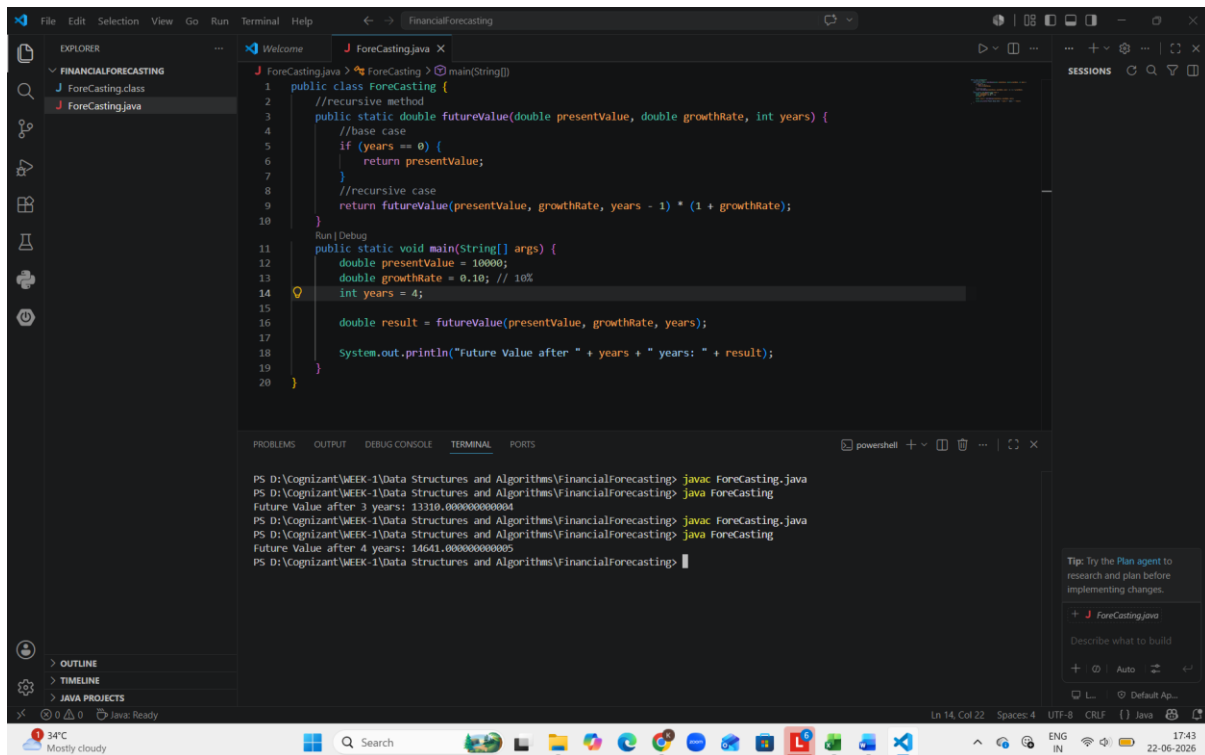
Recursion is a process where a method calls itself to solve multiple subproblems.

It has 2 parts: Base case and Recursive case

->Formula used for calculating future forecast value:

Future Value = Present Value $\times$ (1 + Growth Rate)^{Years}

Output:



```
1 public class Forecasting {
2     //recursive method
3     public static double futureValue(double presentValue, double growthRate, int years) {
4         //base case
5         if (years == 0) {
6             return presentValue;
7         }
8         //recursive case
9         return futureValue(presentValue, growthRate, years - 1) * (1 + growthRate);
10    }
11
12    public static void main(String[] args) {
13        double presentValue = 10000;
14        double growthRate = 0.10; // 10%
15        int years = 4;
16
17        double result = futureValue(presentValue, growthRate, years);
18        System.out.println("Future Value after " + years + " years: " + result);
19    }
20 }
```

```
PS D:\Cognizant\WEEK-1\Data Structures and Algorithms\FinancialForecasting> javac Forecasting.java
PS D:\Cognizant\WEEK-1\Data Structures and Algorithms\FinancialForecasting> java Forecasting
Future Value after 3 years: 13310.000000000004
PS D:\Cognizant\WEEK-1\Data Structures and Algorithms\FinancialForecasting> javac Forecasting.java
PS D:\Cognizant\WEEK-1\Data Structures and Algorithms\FinancialForecasting> java Forecasting
Future Value after 4 years: 14641.000000000005
PS D:\Cognizant\WEEK-1\Data Structures and Algorithms\FinancialForecasting>
```

The time complexity for this algorithm is $O(n)$

->**Methods to optimize the recursive solution:**

Memorization: storing the already computed values to avoid recalculation.

Iterative Approach: using loops instead of recursion.